

Engineering Self-Adaptive Software Systems: From Requirements to Model Predictive Control

KONSTANTINOS ANGELOPOULOS, University of Brighton, UK

ALESSANDRO V. PAPADOPOULOS, Mälardalen University, Sweden

VÍTOR E. SILVA SOUZA, Federal University of Espírito Santo, Brazil

JOHN MYLOPOULOS, University of Ottawa, Canada

Self-adaptive software systems monitor their operation and adapt when their requirements fail due to unexpected phenomena in their environment. This article examines the case where the environment changes dynamically over time and the chosen adaptation has to take into account such changes. In control theory, this type of adaptation is known as Model Predictive Control and comes with a well-developed theory and myriad successful applications. The article focuses on modeling the dynamic relationship between requirements and possible adaptations. It then proposes a controller that exploits this relationship to optimize the satisfaction of requirements relative to a cost function. This is accomplished through a model-based framework for designing self-adaptive software systems that can guarantee a certain level of requirements satisfaction over time by dynamically composing adaptation strategies when necessary. The proposed framework is illustrated and evaluated through two simulated systems, namely, the Meeting-Scheduling exemplar and an E-Shop.

CCS Concepts: • **Software and its engineering** → **Software design engineering**; • **Requirements analysis**; **Computing methodologies** → **Modeling methodologies**;

Additional Key Words and Phrases: Self-adaptive systems, model predictive control, awareness requirements

ACM Reference format:

Konstantinos Angelopoulos, Alessandro V. Papadopoulos, Vítor E. Silva Souza, and John Mylopoulos. 2018. Engineering Self-Adaptive Software Systems: From Requirements to Model Predictive Control. *ACM Trans. Auton. Adapt. Syst.* 13, 1, Article 1 (April 2018), 27 pages.
<https://doi.org/10.1145/3105748>

1 INTRODUCTION

Self-adaptive systems are expected to operate in highly dynamic environments and fulfill multiple goals. When a failure is detected (i.e., a goal is not achieved) due to external disturbances (e.g., high workload or unexpected user behavior), a new configuration is adopted. Unfortunately, composing an adaptation plan to overcome changes in the environment is a challenging task. The main

Authors' addresses: K. Angelopoulos, School of Computing, Engineering and Mathematics, University of Brighton, Watts Building, Lewes Road, Brighton, BN2 4GJ, UK; email: K.Angelopoulos@brighton.ac.uk; A. V. Papadopoulos, Mälardalen University, Höskoleplan 1, 72123, Västerås, Sweden, email: alessandro.papadopoulos@mdh.se; V. E. Silva Souza, Av. Fernando Ferrari, 514 UFES - CT-7 (Dept. Informática), Vitória, ES - 29075-910 - BRAZIL; email: vitorsouza@inf.ufes.br; J. Mylopoulos, School of Electrical Engineering and Computer Science, University of Ottawa, 800 King Edward Avenue, Ottawa ON, CANADA; email: jmylopou@eecs.uottawa.ca.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2018 ACM 1556-4665/2018/04-ART1 \$15.00

<https://doi.org/10.1145/3105748>

obstacle is unsolicited interference from other goals. Therefore, an adaptation strategy A could restore the satisfaction of goal G but fail or worsen goal G' .

Control Theory offers well-developed theoretical and practical frameworks for dealing with systems with multiple parameters (inputs) and assigned with multiple goals (outputs). Some approaches for developing self-adaptive software deal with each goal individually without taking into account interference (Brun et al. 2009; Filieri et al. 2011, 2014; Souza et al. 2012). Others perform reactive adaptation when the failure has already taken place without any provision for the future (Angelopoulos et al. 2014; Cheng et al. 2006; Filieri et al. 2015; Zoghi et al. 2014; Klein et al. 2014) or attempt to anticipate failure. For instance, when the workload grows and a goal fails, additional resources are disposed to the system to overcome the failure. However, if the workload is continuously increasing, it would be wise to dispose more resources than those required for satisfying the goal, in anticipation of future failures. Other approaches (Gaggero and Caviglione 2016; Ghanbari et al. 2014; Kusic et al. 2009; Roy et al. 2011) apply predictive control in the domain of cloud computing to guarantee nonfunctional properties. To our knowledge, despite its effectiveness, software engineers have been reluctant to adopt predictive control for other domains, given the lack of tools and methodologies to model, in a control-theoretic context, software requirements (functional and nonfunctional) and parameters.

In Angelopoulos et al. (2016), we introduced the basic components of Model Predictive Control (MPC) for software systems and how these are related to the requirements of the system-to-be, and also proposed a framework that supports the elicitation of the analytical models required for MPC. Next, we integrated MPC with previous work on requirements engineering for software adaptation in a framework named *CobRA*.¹ Finally, we evaluated our work with a simulation of the Meeting Scheduler exemplar.

This article extends our previous work by applying *CobRA* to a system where the goals are not constrained by clear-cut thresholds but need to be optimized. The absence of thresholds contradicts the principle that a controller continuously performs adaptations in order to ensure that the system's output converges to specific values. Therefore, in this work, (1) we extend our requirements monitoring mechanism in order to capture optimization goals of certain attributes of our target system, e.g., maximize revenue; (2) we propose a systematic approach for representing such goals in the analytical models of the MPC; and (3) we enhance our evaluation by applying *CobRA* to the simulation of an E-Shop in order to demonstrate that our approach can also handle optimization goals.

The rest of the article is structured as follows. Section 2 presents the research baseline for this work. Section 3 describes the basic components of MPC. Section 4 describes the *CobRA* framework. In Section 5, we evaluate *CobRA* using a simulation of the Meeting Scheduler and we compare the results with those of *Zanshin* (Souza et al. 2012), another requirements-based adaptation framework. Finally, in Section 6, we compare related work with our proposal, and Section 7 concludes the article and discusses future directions of this thread of research.

2 BASELINE

Our proposal adopts concepts from both software and systems engineering. The following provides an overview of the baseline from each of these areas.

2.1 Goal Modeling

Goal-Oriented Requirements Engineering (GORE) models elicits requirements as *goals* and analyzes them accordingly. Each goal is iteratively AND/OR-refined to more detailed ones following

¹Control-Based Requirements-oriented Adaptation.

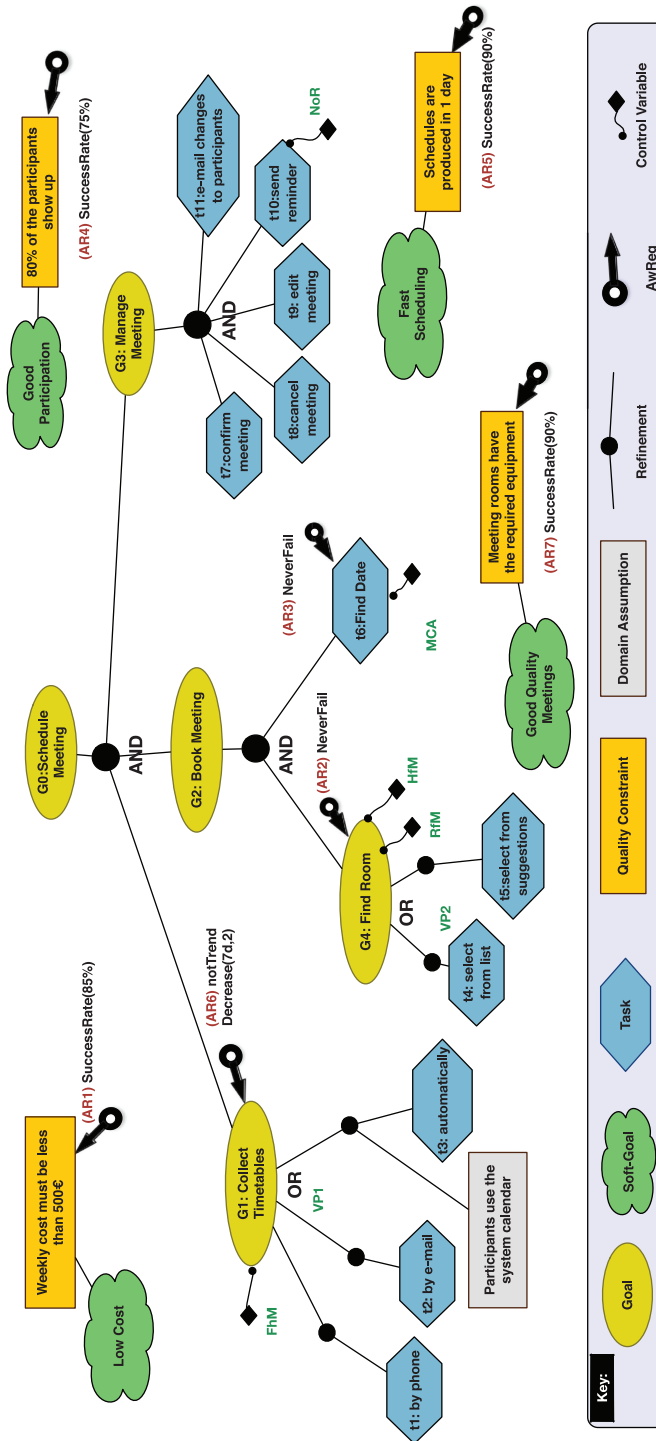
Boolean semantics, until we reach goals that are detailed enough to be operationalized (usually by *tasks*). For the Meeting Scheduler exemplar, the root goal Schedule Meeting (see Figure 1) is refined into three subgoals: Collect Timetables (goal G1), Book Meeting (G2), and Manage Meeting (G3). Goal G1 is fulfilled by either contacting the invited participants by phone (task t1) or by email (t2) or letting the system collect their timetables automatically from the system's calendar (t3). However, the last option is available only if the *domain assumption* that the participants use the system's calendar to register their appointments holds. Next, for goal G2 to be satisfied, goal G4: Find Room must be satisfied by letting the meeting organizer select a room either from the list (t4) or from those suggested by the system (t5). Finally, the meeting organizer should manage the meeting (G3) by confirming its occurrence or cancellation (t7 and t8, respectively), sending reminders to the participants (t9), and notifying them about any change concerning the scheduled meeting (t10).

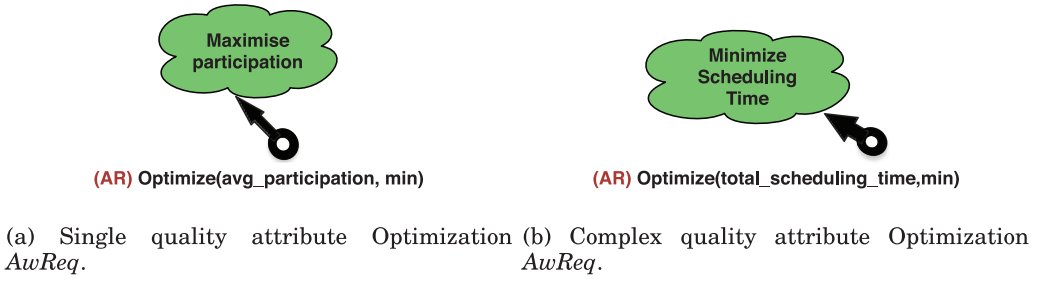
While goals and tasks represent the functional requirements of the system, *soft goals* capture desired nonfunctional properties. Each of the elicited soft goals is quantified by a *quality constraint* that allows reasoning about their fulfillment at runtime. For instance, soft goal Low Cost is satisfied when fewer than 500 euros are spent weekly for organizing meetings. Good Participation is yet another soft goal, satisfied when 80% of the invitees show up for a meeting.

Monitoring and evaluating requirements satisfaction is critical for self-adaptive systems. Following the same line of work as in Angelopoulos et al. (2014), we use *Awareness Requirements* (*AwReqs*) to monitor the success of other requirements. An *AwReq* defines a constraint that triggers adaptation when violated. For example, during an attempt to schedule a meeting, if G4 or t8 fails, a new configuration must be applied (AR2). In the same context, if participation is lower than what stakeholders requested more than 25% of the time, a new adaptation is selected (AR4).

AwReqs can refer to any element of the goal model. When referring to tasks, they determine if the system has performed a specific set of actions successfully. When monitoring a quality constraint, they determine if the system abides by quality requirements. Pointing to goals/soft goals determines if the system has satisfied a requirement. Finally, *AwReqs* that refer to domain assumptions determine if things that were assumed to be true in the environment indeed are true during system operation. *AwReqs* can also refer to other *AwReqs* (meta-*AwReqs*), creating hierarchical feedback loops for adaptation.

Each *AwReq* is associated with variables named *indicators* that measure the degree of fulfillment for a monitored requirement. Indicator values are influenced by two kinds of parameters: (1) *environmental parameters* that are determined by the environment and can't be manipulated by the system and (2) *control parameters* that can be adjusted at runtime by the system (Angelopoulos et al. 2015). Examples of environmental parameters include the number of meeting requests received by the system, as well as participant availability and punctuality. When the number of meeting requests increases significantly, the value of indicator *I2* assigned to AR2 decreases, since it is harder to find a room. Analogously, when participant punctuality or availability decreases, participation is lower and consequently the value of *I4* drops. On the other hand, control parameters are tuned by the adaptation mechanism to correct indicator values that are out of the range of prescribed thresholds. For instance, the number of participants from whom timetables are collected (*FhM*) has an impact on how quickly meetings are scheduled (AR5) and how good participation is going to be (AR4). In the same context, the maximum number of timetable conflicts allowed, *MCA*; the number of reminders sent to the invitees, *NoR*; *VP1* (collecting timetables by phone, email, or automatically) and *VP2* (choosing to find rooms from a list or use a system suggestion); and the numbers of local and hotel rooms, *RfM* and *HfM*, respectively, are control parameters, the values of which affect the outcome of monitored indicators.



Fig. 2. Examples of optimization *AwReqs*.

Formal goal modeling approaches such as KAOS (Dardenne et al. 1993) associate goals with objective functions in order to reason about their satisfaction (Letier and van Lamsweerde 2004). Such objective functions are related to attributes of the system that determine to what level a goal is achieved. In Figure 1, we assume that stakeholders are able to provide clear-cut criteria on whenever a soft goal is satisfied or not. However, it is often the case that stakeholders propose goals that dictate minimization or maximization of certain variables (hereafter optimization goals), e.g., cost, revenue, time, and so forth. We adopt the term quality attribute from Li et al. (2014) to refer to such variables or equations of them. In Figure 2, the optimization goal “Maximize Participation” is associated with the quality attribute average participation (*avg_participation*) in meetings, whereas the optimization goal “Minimize Scheduling Time” is associated to the quality attribute $total_scheduling_time = collection_time + booking_time$, which captures the sum of required time to collect timetables and book a room. Next, we define a new type of *AwReq*, an optimization *AwReq*, which is defined as $Optimize(quality_attribute, optimization\ type\ [min,max])$. The first argument refers to the associated quality attribute and the second defines if this attribute should be minimized or maximized. The value of an indicator of an optimization *AwReq* is equal to its quality attribute and must continuously be optimized. In the following sections, we demonstrate how to use all types of *AwReqs* in order to formulate the requirements satisfaction problem as an optimization problem that is solved with MPC.

In contrast to other kinds of *AwReqs*, the thresholds for the satisfaction of optimization *AwReqs* are not explicit. For minimization goals, we set as threshold the minimum possible value of the associated quality attribute. For example, the minimum value of cost is set to zero. Similarly, for maximization goals, the threshold is set to the maximum possible value of the associated quality attribute. However, if there is no upper limit, the indicator measures the reciprocal value of the quality attribute and the threshold is set to zero. In most cases, the system will never reach these thresholds, although it will continuously search for a configuration to reach them as closely as possible. This behavior complies with the self-optimization property according to which self-adaptive systems must “continually seek opportunities to improve their own performance and efficiency” (Kephart and Chess 2003).

Another kind of requirement included in our approach is an *Evolution Requirement (EvoReq)* (Souza et al. 2013). These apply when certain conditions hold and replace temporarily or permanently other requirements. These changes are applied through actions named *EvoReq* operations. For example, if AR1 constantly fails, probably because the success rate threshold is set too high, it is replaced with a new *AwReq* where the threshold is 75% instead of 85%.

Apart from requirements for the system-to-be, constraints are also imposed on the adaptation process itself, in the form of *Adaptation Requirements (AdReqs)* (Angelopoulos et al. 2014). Examples of such constraints include how much time it should take to restore fulfillment of a failed requirement, or how much a control parameter is allowed to change when its value is modified.

2.2 Dynamic System Modeling

In our previous work (Angelopoulos et al. 2014; Souza et al. 2011), qualitative relations have been used to model the relation between control parameters and indicators. Often, using qualitative adaptation is a necessity, given the lack of quantitative models for software systems. However, in many cases, a sufficiently accurate quantitative dynamic model can be obtained through system identification techniques (Ljung 1999) and can be used for control design. Letting $u(t) \in \mathbb{R}^m$ be the vector of control parameter values at time t and $y(t) \in \mathbb{R}^p$ be the vector of indicators, their respective dynamic relation is described as

$$y_i(t) = \sum_{j=1}^p \sum_{k=1}^{n_y} \alpha_{ijk} y_j(t-k) + \sum_{j=1}^m \sum_{k=1}^{n_u} \beta_{ijk} u_j(t-k) \quad (1)$$

for all $i = 1, \dots, p$, and with $\alpha_{ijk} \in \mathbb{R}$, $\beta_{ijk} \in \mathbb{R}$. The quantitative dynamic model in Equation (1) relates the values of the indicator y_i at time t with past values of all the indicators—accounting for possible mutual influences of the indicators—and with past values of control parameters. For example, $I1$ might achieve a high value because of good management of hotel room assignments or because of the constant failure of $I2$. The reason is that if meetings fail to be scheduled, no rooms are reserved and consequently the cost of meetings remains low. Such implicit relationships among indicators can be captured by Equation (1) to guide the adaptation process. Notice that if some of the mentioned variables are not influencing the value of the indicator $y_i(t)$, then the corresponding parameters are simply zero. An equivalent and more compact representation of this relation is the discrete-time state-space dynamic model:

$$\begin{cases} x(t+1) = Ax(t) + Bu(t) \\ y(t) = Cx(t), \end{cases} \quad (2)$$

where $x(\cdot)$ is a vector named the *dynamic state* of the model. While for physical systems the state $x(\cdot)$ is typically associated with meaningful physical quantities, in general, the state can be just an abstract representation of the system, and it is not necessarily measurable. The values of the matrices (A, B, C) fully describe how the inputs dynamically affect the outputs of the system, and these matrices are the outcome of the System Identification process.

The analytical model of Equation (1) shows that the system's output might be related to past outputs and control inputs. Indicators related to aggregate *AwReqs* (Souza et al. 2011) express success rates over time about the satisfaction of an associated goal, and therefore, their current values are naturally bound to their past values and to the values of *AwReqs* that produced them. These are *dynamic systems* in Control Theory. In case no relation with the past behavior of the indicators and of the control inputs is present, A is a matrix with all zero elements, and the system is just mapping inputs to outputs with the *static* relation

$$y(t) = CBu(t-1).$$

Therefore, the model of Equation (2) accounts for both dynamic and static systems.

Equation (2) can be used to design a control system able to adjust the values of every control parameter, in order to make each indicator converge to the value prescribed by an *AwReq* threshold—under the assumption that the set of chosen control parameters is able to drive the system to the prescribed goals. In contrast to qualitative adaptation, such quantitative models allow one to handle conflicts with precision. For example, an increase of the control parameter *MCA* results in an increase of $I3$, as it becomes easier to find a commonly agreed-upon timeslot for the meeting, but the participation might drop and consequently $I4$ is decreased. The analytical model can prevent the adaptation mechanism from decreasing $I4$ excessively. Performing such tradeoffs on a daily basis while taking into account priorities among indicators based on their business value (higher-priority indicators should converge faster than less important ones) and preferences among control

parameters (e.g., increasing RfM is preferred to increasing HfM) is a complex process. In the next section, we present a control-theoretic approach in order to efficiently implement this process and maintain an equilibrium among conflicting goals.

Notice that software systems are often nonlinear, nonsmooth, or even discontinuous (Papadopoulos et al. 2015), and the linear dynamics that are identified are just an approximation of the actual dynamics of the system. Linearizing nonlinear dynamics is common practice in most control applications (Åström and Murray 2010), and if the control design is carried out correctly, the feedback loop, possibly jointly with some learning strategies, is in charge of handling model inaccuracies and neglected dynamics in a robust way (see, e.g., Kothare et al. (1996)). The main challenge when dealing with computing systems is that obtaining reliable dynamic models is really complicated, since there are no laws of physics, and it is application dependent (Papadopoulos et al. 2015). This work is part of a long-term research line whose aim is to automatize the decision-making policy design from the elicitation of the system requirements to the actual implementation. The use of model identification techniques eases the construction of the dynamic model and could be easily included in an automated tool.

3 MODEL PREDICTIVE CONTROL

Based on the dynamic model of Equation (2), different control strategies can be designed. We here extend our previous work (Angelopoulos et al. 2015) and present a *receding horizon* MPC (Camacho and Bordons 2004; Maciejowski 2002) that is able to manage the achievement of multiple conflicting goals by means of multiple control parameters. When the controller is complemented with a Kalman filter (Ljung 1999), it can learn online how to adapt the controller to the system's behavior, overcoming inevitable inaccuracies coming from dynamics not captured from Equation (2) and unknown disturbances acting on the system.

MPC is a control technique that formulates an optimization problem to use a set $u(\cdot)$ of control parameters (actuators) to make a set of indicators $y(\cdot)$ achieve a set of goals $y^\circ(\cdot)$ over a prediction horizon H . At every control instant t , the values of the control parameters $u^\star$ are obtained by minimizing a cost function J_t , subject to given constraints. The optimization problem includes a prediction of the future behavior of the system based on the dynamic model in Equation (2). An obtained solution is therefore a plan of the future control parameter values $u^\star = [u_t^\star, u_{t+1}^\star, \dots, u_{t+H-1}^\star]$ over the prediction horizon. This planning is especially needed in the case of delay in the effects of changes of control parameters. For example, increasing the number of hotel rooms requires approval by the administration council that meets only every 2 days. Hence, the adaptation mechanism must be aware of when changes to control parameters impact on the indicators and make look-ahead plans.

According to the receding horizon principle, only the first computed value $u_t^\star$ is applied to the system, i.e., $u(t) = u_t^\star$. The reason is that for real-world systems, it is impossible to derive perfect models that describe their dynamic behavior. Therefore, the plan must be corrected at each step and the horizon recedes by 1 unit. Another reason the plan might fail is a change in the external disturbances (e.g., system workload). In other words, the plan would have been followed as is only if a perfect model were available and no disturbances were present, which in practice is impossible. To tackle this obstacle, at the next control instant, a new plan is computed according to the new measured values of the indicators. This accounts for modeling uncertainties and possible unpredictable behaviors of the system that are not captured by Equation (2).

3.1 Formal Description

In order to present the underlying rationale of the MPC, it is convenient to rewrite the dynamic model in Equation (2) in an “augmented velocity form”:

$$\begin{aligned}
\overbrace{\begin{bmatrix} \Delta x(t+1) \\ y(t) \end{bmatrix}}^{\tilde{x}(t+1)} &= \overbrace{\begin{bmatrix} A & 0_{n \times p} \\ C & I_{p \times p} \end{bmatrix}}^{\tilde{A}} \overbrace{\begin{bmatrix} \Delta x(t) \\ y(t-1) \end{bmatrix}}^{\tilde{x}(t)} + \overbrace{\begin{bmatrix} B \\ 0_{p \times m} \end{bmatrix}}^{\tilde{B}} \Delta u(t) \\
y(t) &= \overbrace{\begin{bmatrix} C & I_{p \times p} \end{bmatrix}}^{\tilde{C}} \overbrace{\begin{bmatrix} \Delta x(t) \\ y(t-1) \end{bmatrix}}^{\tilde{x}(t)}.
\end{aligned} \tag{3}$$

Here, $\Delta x(t) = x(t) - x(t-1)$ is the state variation and $\Delta u(t) = u(t) - u(t-1)$ is the control increment. The output of the system $y(t)$ is unchanged but is now expressed with respect to the state variations $\Delta x(t)$ and not with respect to the state values $x(t)$. The new dynamic model in Equation (3) is used as a prediction model over a finite horizon H . This means that the controller will use it to predict what values of the states and of the indicators are going to be after H time steps from the current one. The MPC controller minimizes the cost function

$$J_t = \sum_{i=1}^H \left([y_{t+i}^\circ - y_{t+i}]^T Q_i [y_{t+i}^\circ - y_{t+i}] + [\Delta u_{t+i-1}]^T P_i [\Delta u_{t+i-1}] \right),$$

where $Q_i \in \mathbb{R}^{p \times p}$ and $P_i \in \mathbb{R}^{m \times m}$ are symmetric positive semidefinite weighting matrices that respectively represent the importance of the distance between the goals and the current values and the “inertia” in changing the values of the actuators. In particular, Q_i is a diagonal matrix that contains the values of the set of weights that can be obtained by applying the Analytical Hierarchy Process (AHP) (Karlsson and Ryan 1997), in which the stakeholders perform pairwise comparisons to prioritize the elicited goals. This means that when not all the goals are simultaneously feasible (e.g., because one conflicts with another), the controller will favor the satisfaction of the goals with the higher weights. The matrix P_i captures preferences over control parameters. When a control parameter is requested not to change its value frequently, the assigned weight must be relatively smaller than most of the weights of the other control parameters. In the following, we will consider the weight matrix Q as $Q := Q_1 = Q_2 = \dots = Q_H$, and the weight matrix P as $P := P_1 = P_2 = \dots = P_H$; i.e., the weight matrices are considered to be constant along the prediction horizon.

The resulting MPC optimization problem can be written as follows:

$$\begin{aligned}
&\text{minimize}_{\Delta u_{t+i-1}} J_t \\
&\text{subject to } u_{\min} \leq u_{t+i-1} \leq u_{\max}, \\
&\quad \Delta u_{\min} \leq \Delta u_{t+i-1} \leq \Delta u_{\max}, \\
&\quad \tilde{x}_{t+i} = \tilde{A} \cdot \tilde{x}_{t+i-1} + \tilde{B} \cdot \Delta u_{t+i-1}, \\
&\quad y_{t+i-1} = \tilde{C} \cdot \tilde{x}_{t+i-1}, \\
&\quad i = 1, \dots, H, \\
&\quad x_t = x(t).
\end{aligned} \tag{4}$$

This formulation is equivalent to a convex Quadratic Programming (QP) problem (Maciejowski 2002). The problem has time complexity $\mathcal{O}(H^3 m^3)$ (Wang and Boyd 2010). A solution to the problem consists of a plan of optimal future Δu_{t+i-1}^* , $i = 1, \dots, H$, but only the first one is applied, i.e., $\Delta u(t) = \Delta u_t^*$, as we explained earlier. The new control signal is then

$$u(t) = u(t-1) + \Delta u(t). \tag{5}$$

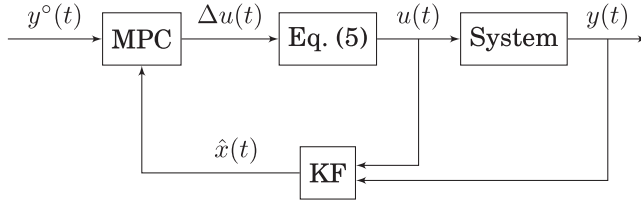


Fig. 3. Reference control scheme for CobRA.

The MPC strategy assumes that the state of the system is measurable, but in many cases this is not possible. Indeed, since there is often no correlation with physical quantities, it is impossible to give a meaningful interpretation to $x(t)$, and hence it is impossible to measure. However, based on the dynamic model in Equation (2), it is possible to estimate its value measuring the values of $y(t)$ and $u(t)$. To accomplish this, we here use a Kalman filter (KF) that finds an estimate $\hat{x}(t+1)$ of the state $x(t+1)$, measuring the applied control signal $u(t)$ and the output $y(t)$:

$$\begin{aligned} \hat{y}(t) &= C\hat{x}(t) \\ \hat{x}(t+1) &= A\hat{x}(t) + Bu(t) + K(y(t) - \hat{y}(t)). \end{aligned} \quad (6)$$

Note that the variables of the KF are commonly denoted by a “hat,” i.e., $\hat{x}(k)$ and $\hat{y}(k)$, to distinguish them from the variables of the dynamic model in Equation (2). Based on the state estimate $\hat{x}(t)$, the KF shown in Equation (6) computes an estimate of the output $\hat{y}(t)$ to measure the difference between the predicted value $\hat{y}(t)$ and the real value $y(t)$. The value of K , called *Kalman gain*, weights the discrepancy between the predicted value $\hat{y}(t)$ and the real value $y(t)$, adjusting the dynamics of the KF (Ljung 1999). The estimate $\hat{x}(t)$ can be used, in place of $x(t)$, to solve the optimization problem in Equation (4).

The adopted KF has a twofold functionality. First, as we just described, based on the dynamic model in Equation (2), it computes a state estimate $\hat{x}(t)$ that the MPC uses to compute the next control action. Second, it is adapting the state estimate to the actual behavior of the system. This is relevant for a number of reasons: the controlled system may change its behavior over time, there might be unpredictable disturbances acting on the system, or the system is not following the linear dynamics of the dynamic model in Equation (2). In all cases, the KF is adapting online the choice of the estimate $\hat{x}(t)$, returning a value that is compatible with the input-output behavior of the running system, as if it was described exactly by the dynamic model in Equation (2) (Ljung 1999).

The block diagram for the resulting control scheme is represented in Figure 3.

3.2 Formal Guarantees

Applying Control Theory to software systems provides a set of formal guarantees about the quality of the adaptation process (Filieri et al. 2017). The MPC adopted in this work belongs to a class of controllers named *optimal controllers*, since the computation of control decisions is based on the solution of an optimization problem. In particular, the MPC accounts for model predictions in order to make optimal adaptation plans with respect to system requirements and compliance to the requirements about the adaptation process itself (Angelopoulos et al. 2014).

The formal guarantees for the MPC are as follows. First, it is possible to ensure that all the goals are reached, subject to actuator constraints; i.e., there exists a value of the actuators within the given constraints specified in the optimization problem in Equation (4) that allow the system to reach its goals. If this is not the case, due to the optimal nature of the controller, the MPC finds a configuration for the actuators that minimizes the distance between the indicators and the

goals. Such a distance depends on the chosen weights for each indicator in the cost function of the optimization problem in Equation (4).

Furthermore, since the cost function accounts for a time horizon, it is possible to guarantee that the convergence time is minimal. The dynamic model in Equation (2) relates control parameters and indicators including the dimension of time. Therefore, the adaptation mechanism is able to drive the system to the goals as soon as possible, as specified by the cost function of the optimization problem in Equation (4). Moreover, the optimization problem in Equation (4) can be easily extended in order to account for additional constraints, such as, for example, constraints on the indicators (Camacho and Bordons 2004). *AwReqs* and *AdReqs* impose such constraints over the elicited goals and the adaptation process, respectively, which must be taken into consideration when a new adaptation plan is produced.

The MPC formulation is well suited for also addressing real-time issues and has been applied to various domains, such as aircraft control (Qin and Badgwell 2003; Hartley et al. 2014a). Since the proposed solution requires a solution to an optimization problem at each control instant, it is critical to discuss such possible real-time issues. In many cases, in fact, the time required for computing the value of the next control action might be longer than the time between two subsequent control actions. In order to overcome this challenge, there is significant literature in the control community on how to implement fast solvers (Jerez et al. 2012; Giselsson 2014), especially for embedded systems (Jerez et al. 2014), possibly also codesigning a dedicated hardware for the solution in case of time-critical systems (Hartley et al. 2014b). An overview on the matter can be found in Zeilinger et al. (2014).

In many cases, such advanced algorithms are not required when dealing with software components, and for the most time-critical applications some modification to the control problem can help in reducing the complexity. For example, one way to reduce the complexity is to set $\Delta u_{t+1} = \Delta u_{t+2} = \dots = \Delta u_{t+H-1} = 0$ and let the optimization problem decide only the value for Δu_t , i.e., the one that will actually be applied to the system. This modification reduces the complexity to $O(m^3)$.

Another way to deal with real-time issues is to exploit simple properties of interior point algorithms. In fact, the solution is obtained in a fixed amount of steps with an iterative method. The current solution is always a suboptimal yet feasible solution to the optimization problem. This means that if the iterative method did not converge before a new control action is required, it can be forced to stop and return the current suboptimal solution. This allows the controller to fulfill real-time deadlines.

Finally, another possibility to deal with real-time deadlines is exploiting the proactive nature of the MPC. As we mentioned earlier, the MPC is computing at each iteration step a plan of future actions Δu_{t+i-1} , $i = 1, \dots, H$; then, according to the receding horizon principle, only the first one is applied, i.e., $\Delta u(t) = \Delta u_t^*$. Assuming that at the next control instant the solver takes more time to converge and that a new control action is required before the optimal solution is found, one can store the previously computed plan and apply the second control action, i.e., $\Delta u(t+1) = \Delta u_{t+1}^*$. This is obviously suboptimal, since it neglects the last information about the measured output, but it is able to fulfill the real-time deadlines.

4 APPROACH

Our approach involves two phases: design and runtime. During the first phase, all models required for the MPC controller's synthesis and tuning are elicited, whereas during the second phase, the controller is deployed in our adaptation framework and adjusts the control parameters of the target system when required.

Table 1. Reference Goals

<i>AwReq</i>	$y^\circ(\cdot)$
AR1	$y_1^\circ(\cdot) = 85$
AR2	$y_2^\circ(\cdot) = 100$
AR3	$y_3^\circ(\cdot) = 100$
AR4	$y_4^\circ(\cdot) = 75$
AR5	$y_5^\circ(\cdot) = 2$
AR6	$y_6^\circ(\cdot) = 90$
AR7	$y_7^\circ(\cdot) = 90$

Table 2. *EvoReqs* Operations

<i>AwReq</i>	<i>EvoReq</i> operation
AR1	1. Relax (AR1,AR1'_75) 2. Strengthen (AR1,AR1'_85)
AR2	Relax (AR2,AR2'_90)
AR3	Relax (AR3,AR3'_90)
AR4	1. Wait (3 days) 2. Relax (AR4,AR4'_75)
AR5	Replace (AR5,AR5'_3)
AR6	Wait (3 days)
AR7	Wait (2 days)

4.1 Design Phase

Our approach starts with the elicitation of all kinds of requirements about the target system. When all goals are refined, *AwReqs* are assigned to those that are considered most critical and prone to failure. An *AwReq* AR_i defines a reference goal $y_i^\circ(\cdot)$ for the system's output. For instance, Table 1 enlists all the reference goals for the Meeting Scheduler exemplar.

As we mentioned earlier, the constraints imposed by *AwReqs* are not always feasible or might become infeasible in the future. For instance, prices of hotel rooms rise every year and consequently $I1$ will fail more often as time passes. In addition, during summer, prices are usually higher. Hence, stakeholders could accept a lower success for $I1$ (in other words, $y_1^\circ(\cdot) < 85$). At this step of the design phase, the domain experts, along with the stakeholders, analyze and evaluate such conditions and specify *EvoReqs* for the system-to-be. The *EvoReqs* operations defined for the *AwReqs* of the Meeting Scheduler are presented in Table 2.

When summer season begins and hotel prices are higher, the first *EvoReq* operation is triggered, relaxing the reference goal from 85% to 75%. The second *EvoReq* operation is triggered when summer season ends and the threshold is restored to its previous value. Similarly, when AR2 and AR3 fail for more than 2 days in a row, the reference goals are relaxed for a week.² In case of AR5, when goal $G1$ tends to fail more than 2 times/week, the constraint is permanently replaced by 3 times/week. Finally, when AR6 and AR7 fail for more than 2 days, the adaptation mechanism ignores them for 3 and 2 days, respectively.

²The relaxation duration and the triggering condition are prescribed by the stakeholders.

Table 3. Indicator Priorities

Indicator	Priority
$I1$	0.15
$I2$	0.3
$I3$	0.3
$I4$	0.06
$I5$	0.2
$I6$	0.05
$I7$	0.04

Table 4. Control Parameter Weights

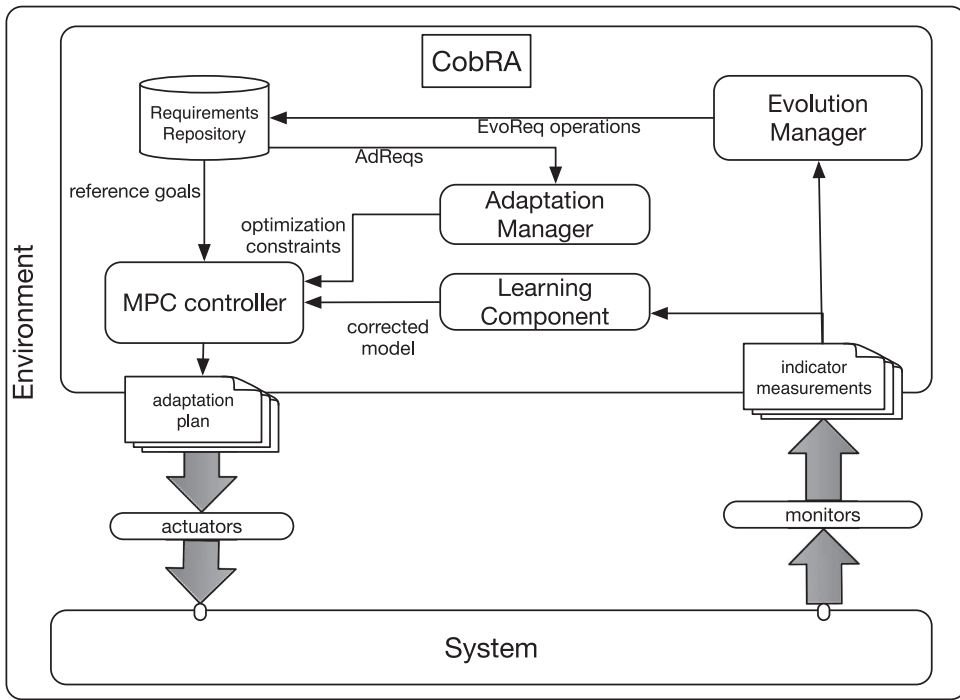
Control Parameter	Weight
FhM	1
MCA	1
RfM	1.2
HfM	0.6
NoR	1.2
$VP1$	0.8
$VP2$	1.4

Next, by applying AHP, weights are elicited for each indicator to capture their relative importance. As a rule of thumb, indicators assigned to functional requirements have higher priority compared to nonfunctional ones. These weights are the values of matrix Q of the cost function. The controller, through the optimization function, finds an equilibrium for every goal, putting more effort on fixing the most important ones. As for the control parameters, their weights are empirically elicited, assigning lower weights to the control parameters we want to be tuned less often. These weights are the values of matrix P of the cost function. In our exemplar, for instance, increasing the number of rooms RfM is preferred over HfM since it is a less costly solution, does not require any authorization, and therefore, takes effect immediately. The elicited priorities for the indicators of Meeting Scheduler and the weights for control parameters are shown in Table 3 and Table 4, respectively.

The last set of requirements to be elicited are the *AdReqs*. These requirements impose constraints to the adaptation process itself. For the particular case of MPC, an *AdReq* specifies the receding horizon of the controller and, consequently, how far in the future the adaptation plan should target. Other *AdReqs* might refer to the magnitude of allowed change of control parameters. For instance, HfM cannot be increased more than 5 units each time.

Finally, a quantitative model such as the one in Equation (2) must be derived. Given the absence of laws of nature over this particular domain, we ran a long simulation of the Meeting Scheduler system, during which the control parameters change often and both control input and output are recorded. With the aid of Matlab and System Identification toolbox,³ we estimate the analytical model of the system. Even if the system-to-be cannot be simulated accurately, the model can be

³<http://it.mathworks.com/products/sysid/?requestedDomain=www.mathworks.com>.

Fig. 4. Scheme of the *CobRA* framework.

improved later on, when the real system is deployed, by means of a learning mechanism during the runtime phase.

4.2 Runtime Phase

When the design phase is completed and the system is implemented, the *CobRA* framework can be deployed and play the role of the adaptation mechanism. *CobRA*, depicted in Figure 4, has five main components. The monitors and the actuators that integrate *CobRA* with the target system are application specific and must be implemented by the designers of the system.

Requirements repository. This repository stores all the models produced during the design phase and provides information to the other components of the framework when requested.

Evolution manager. This component analyzes the logs provided by the monitors in order to identify conditions that would trigger *EvoReq* operations. If a requirement is replaced either permanently or temporarily, it updates the requirements repository.

Adaptation manager. This component translates *AdReqs* to constraints for the optimization problem of Equation (4). Such constraints are related to the maximum allowed decrease or increase of a control parameter in a single step and the weights of all indicators and control parameters (matrices Q and P).

Learning component. Black-box system identification does not always provide precise models about the system's behavior. Therefore, we include in our framework a learning component that, based on the applied changes and the outcome values of that indicator that occurred as a result of these changes, revises the control law to adapt to changes of the behavior of the system. More specifically, this component is an implementation of the Kalman filter as described in the previous section.

MPC controller. The details of this component have been discussed in the previous section. Summarizing its functionalities, the MPC controller requests the requirements repository for the reference goal $y^\circ(\cdot)$ of each indicator monitored. It then calculates the distance of each indicator from its respective reference goal and composes an adaptation plan that minimizes every distance, taking into account the indicator priorities in order to restore equilibrium, subject to the given constraints on the control parameters. The plan includes changes to control parameters in a pre-defined horizon. For example, the indicators of the Meeting Scheduler are evaluated daily and the plan includes values for control parameters so that indicators minimize their distance from $y^\circ(\cdot)$ for the next 3 days. If 2 days after the plan is applied the result is not what was expected, e.g., because the number of meetings constantly grows, the controller produces a new plan that tries to anticipate future failures in a receding horizon fashion.

The iterative adaptation process with *CobRA* includes the following steps:

- **Step 1:** The monitors collect the measurements of all the indicators of the system.
- **Step 2:** The *Evolution Manager* examines if any event that would trigger an *EvoReq* operation is present and, in that case, updates the evolved requirement in the Requirements Repository.
- **Step 3:** The Adaptation Manager provides the MPC controller with the weights for the indicators and control parameters, as well as constraints for the optimization problem.
- **Step 4:** The Learning Component provides the MPC with a corrected model of the system based on the recent measurements.
- **Step 5:** The MPC controller, given the current reference goals provided by the Requirements Repository and the corrected model, produces a revised adaptation plan with the target indicator value to converge to the reference goal within the prediction horizon.
- **Step 6:** The actuators apply the first step of the plan to the system.

It is important to mention that if a new requirement is introduced or an older one is removed, the design phase must be repeated in order to derive a new analytical model.

5 EVALUATION

In the previous sections, we provided the basic background for the structure and functionalities of an MPC controller. We also presented the *CobRA* framework, which exploits stakeholder goals and uses an MPC controller to compose dynamically adaptation plans when requirements are not met. In this section, we evaluate and compare *CobRA* with *Zanshin* (Souza et al. 2012), which also has stakeholder requirements as its baseline for adaptation and adopts concepts from Control Theory.

5.1 Methodology

We have conducted our experiments with a simulation of the Meeting Scheduler exemplar⁴ and another for the E-Shop exemplar,⁵ both implemented in Python and run on a computer with an Intel i5 processor at 2.5GHz and 16GB of RAM. We stress-tested both our simulations for long periods while modifying all control parameters in order to cover the whole range of their potential values. The result of this process is a log file for each application with all the values of the inputs and outputs of the system at every step. Then, we executed a Matlab procedure from the Matlab System Identification Toolbox in order to estimate an analytical model that describes each system's behavior as it is described in Section 2.

⁴https://gitlab.com/konangelop/it.unitn.disi.konangelop.simulations.meeting_scheduler_v2.git.

⁵[git@gitlab.com:konangelop/E-Shop-TAAS.git](https://gitlab.com/konangelop/E-Shop-TAAS.git).

After acquiring the system's quantitative model, we stress-tested the simulation by modifying various environmental parameters, such as the system's workload. At this phase, we tuned the controller by modifying the weights of the outputs and the inputs. If an indicator, especially a not very important one, constantly overshoots, its weight must be reduced. Similarly, if there is a control parameter that we do not wish to change often, such as the number of hotel rooms available, its associated weight is increased. As a rule of thumb, the user must keep the weight values in the same magnitude. Moreover, the order of the modified weights of the indicators must be compliant with the one the stakeholders provided. This process is iterative and requires multiple executions of the simulation. When the MPC controller reaches a desired behavior, the tuning phase is completed.

Zanshin, as opposed to *CobRA*, does not involve any quantitative models, only qualitative relations between inputs and outputs based on human expertise. For example, it is known by the domain expert that by increasing *MCA*, the value of *I3* increases. For our experimentation, we used the default adaptation algorithm of *Zanshin* as described in Souza et al. (2012). When a failure arrives, *Zanshin* randomly selects a control parameter that will improve this failure and increases or decreases it by a predefined amount. Therefore, we provided such qualitative information to *Zanshin* based on a previous study of the Meeting Scheduler (Angelopoulos et al. 2014; Souza et al. 2011).

For the evaluation and the comparison of the two frameworks, we put the simulated systems under a stress test and compare the behavior of the outputs in each case. We also compare the values of the cost function described in Section 3 through time for both frameworks, comparing which minimizes it most. The selection of *Zanshin* for the purposes of our evaluation is based on two reasons. First, it uses the same requirements-based monitoring mechanism (*AwReqs*) as *CobRA*, and therefore, customization of the adaptation problem was not required. Second, *Zanshin* decides adaptation plans based on qualitative information provided by domain experts, while *CobRA* uses an automatically derived quantitative model that captures the dynamics of the system.

5.2 Meeting Scheduler

The Meeting Scheduler application receives a number of meeting requests daily. Once the timetables are collected, a date for each meeting must be found. The result of the finding-a-date process is pseudorandom, given that it depends on control and environmental parameters that change based on stochastic processes we have encoded in the simulation. For instance, as the availability of the participants drops, the the goal *Find Date* will fail more often. Similar pseudorandomness has been encoded for other goals such as *Find Room* and *High Participation*. For our experiment, we run the simulation for 60 steps (simulation days), during which the number of meeting requests gradually increases and then decreases along with participants' availability. Due to space limitations, we present the results only for indicators *I1*–*I4*.

Figure 5 depicts the values of the indicators at each step of the simulation. In particular, we run the simulation using multiple different values for the horizon of the MPC. As the number of meetings grows, the cost for the system increases as well, resulting in a decrease of indicator *I1*. However, *CobRA* manages to recover by preferring local rooms over hotel rooms, as can be seen in Figure 6, whereas *Zanshin* fails to recover from the failure. Moreover, *CobRA* adapts faster for higher values of the horizon. This is also captured by Figure 6, where the pace of decreasing hotel rooms and increasing local rooms is faster for higher values of the MPC's horizon. Concerning indicator *I2*, *CobRA* converges almost immediately, whereas *Zanshin* requires considerably more time. The reason for the delay is that *Zanshin* increases its control parameters by a fixed amount rather than basing it on the magnitude of failure as *CobRA* does. In the case of *I3*, the human expertise provided to *Zanshin* matched the identified relation we derived experimentally for *CobRA*,

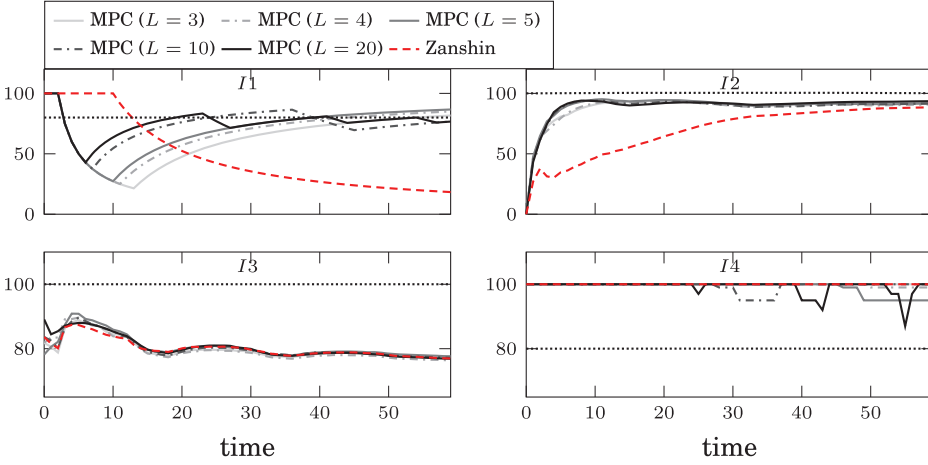


Fig. 5. Indicator measured values for the Meeting Scheduler exemplar.

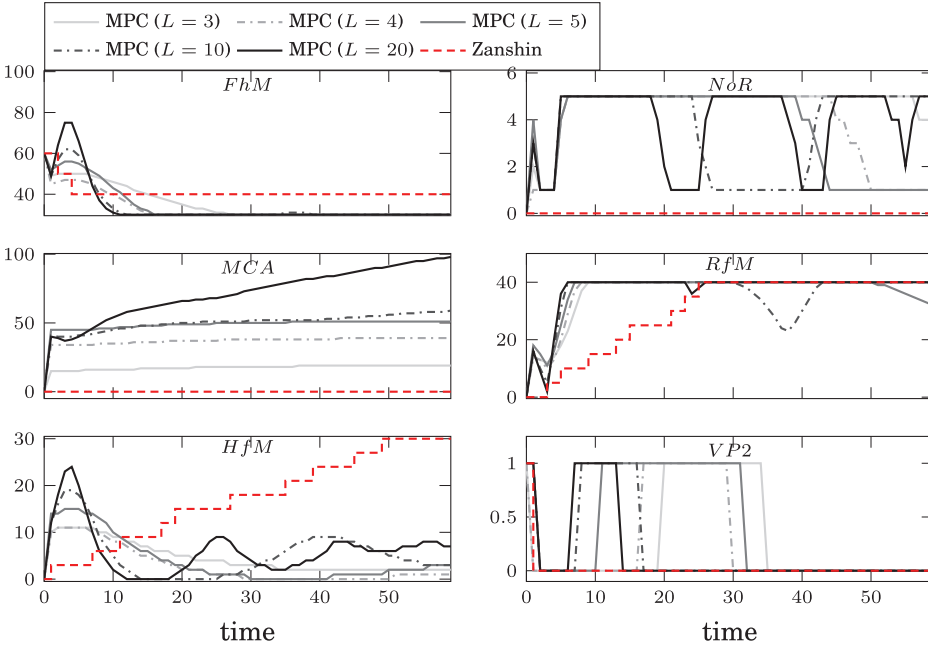


Fig. 6. Control parameter values for the Meeting Scheduler exemplar.

since the two frameworks achieved almost identical values. Both frameworks decreased the value of FhM , which results in decreasing the participation if the punctuality of the participants drops. This is what happens during the period between the day 30 and 40, which causes *Zanshin* to fail to reach the expected threshold for indicator $I4$. On the other hand, *Cobra* responds to the decrease of punctuality by increasing NoR more than the latter. Finally, for $I2-I4$, the impact of the horizon value is not significant since these indicators are changing at a slower pace than $I1$, and therefore, even with a short prediction horizon, the system adapts in the same manner.

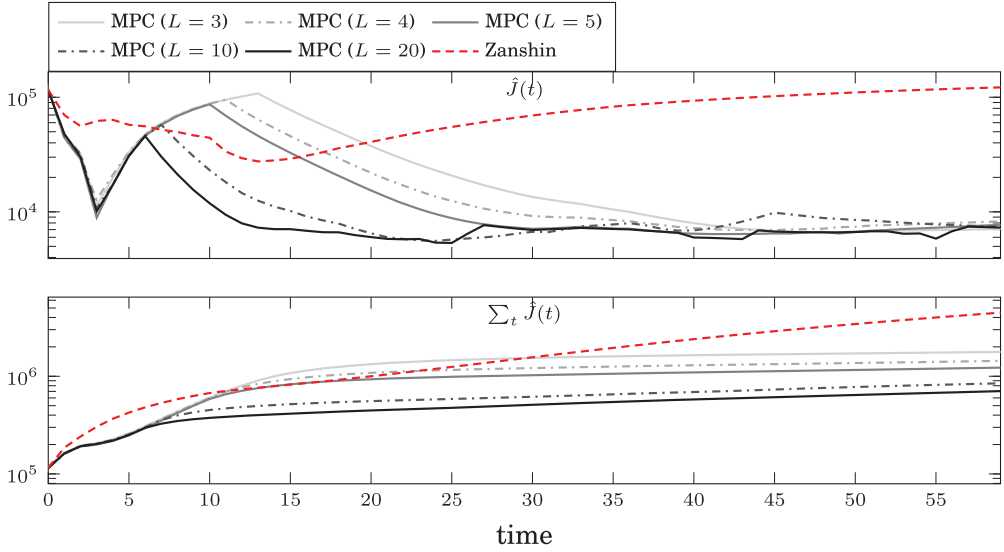


Fig. 7. Adaptation cost for the Meeting Scheduler exemplar associated with the cost function J for the considered adaptation strategies. The top graph shows the instantaneous cost, while the bottom graph shows the cumulative cost.

CobRA handles the adaptation problem, i.e., continuously search for values for the system's control parameters in order to minimize all control errors with respect to the priority of the associated indicator and the effort required to change these values, as a multiobjective optimization problem. Therefore, as a metric to evaluate our approach, we use the the cost function $\hat{J}(t) = (y^\circ(t) - y(t))^T Q (y^\circ(t) - y(t)) + \Delta u(t)^T P \Delta u(t)$. The value of this cost function is calculated from the measured values of the indicators and the changes performed over control parameters. In Figure 7, we present the individual value of the cost function at each step of the simulation at the top and the cumulative cost $\sum_t \hat{J}(t)$ at the bottom. $\hat{J}(t)$ captures business value loss because of failing indicators and adaptation costs for changing control parameters. *CobRA* minimizes more at each step of the cost function, and by the end of the simulation it produces more stable results. On the other hand, *Zanshin*'s adaptation results in higher losses at most steps, while the accumulated value of the cost function is growing monotonically. Furthermore, for higher values of the horizon, *CobRA* presents better results compared to short horizons by minimizing the cost function more. In this example, we can see that different values of the time horizon provide some differences in the obtained system behavior. In particular, we can observe that for higher values of the horizon, the system exhibits better performance. Recall, however, that the solution of the optimization problem for the MPC formulation scales as L^3 , and very high values of the prediction horizon might lead to intractable computation times. When tuning the controller, one then needs to find a tradeoff between computation time and obtained performance, also considering that the cost associated with the minimized cost function monotonically decreases while increasing the prediction horizon. The minimization of the cost over the simulation is highlighted in the cumulative cost shown in the bottom graph of Figure 7.

5.3 E-Shop

The E-Shop application consists of three simulated components: (1) a Load Balancer, (2) a server pool, and (3) a traffic generator. The traffic generator produces a trace of arrivals per minute for the

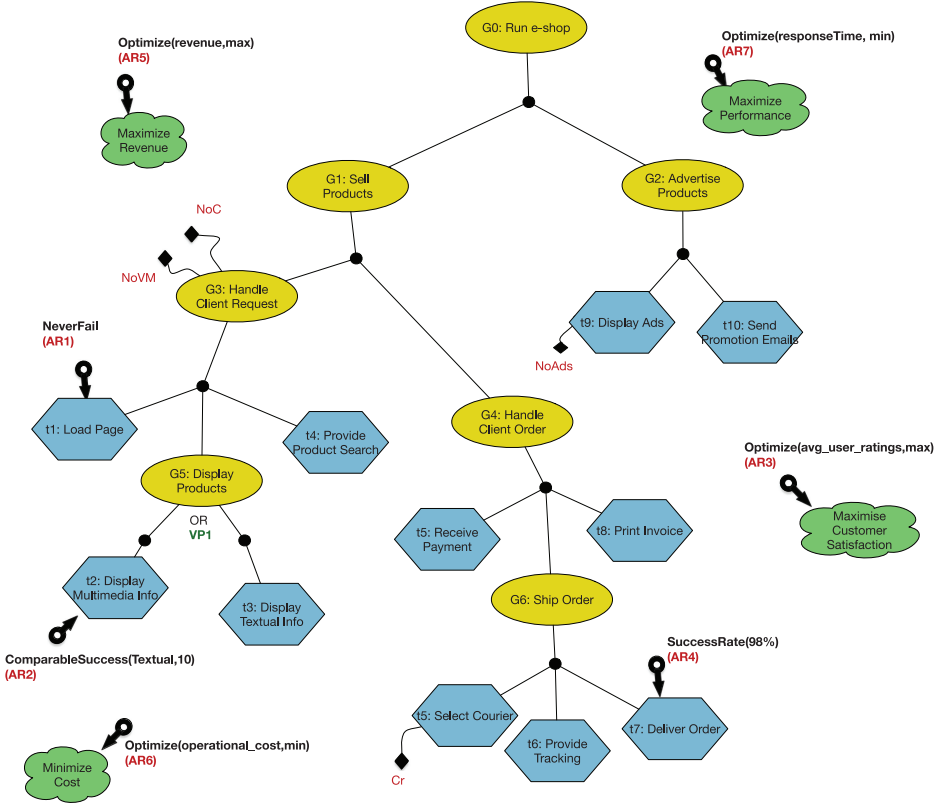


Fig. 8. E-Shop goal model.

entire duration of the simulation, which is decided by the user. Every minute the Load Balancer receives new arrivals and distributes them equally to the active servers of the pool. The purpose of an E-Shop is to sell and advertise products (G1 and G2, respectively), as depicted in Figure 8. Satisfying G1 requires handling requests (G3) for loading the E-Shop’s page, providing a product search functionality (t4) that allows customers to better navigate through the catalog, and displaying the selected products (G5) either including multimedia information, e.g., videos (t2), or showing only textual description (t3). Next, the customer orders must be handled (G4) by receiving the payment (t5), shipping the order (G6), and printing an invoice (t8). For shipping an order, a courier company must be selected (t5), provide tracking to the customer (t6), and eventually deliver the order (t7). Finally, the E-Shop also displays ads (t9) and sends advertisement emails to its customers (t10).

We have identified seven *AwReqs* for the E-Shop application. First, the servers must never fail to load the website’s page (AR1). Next, the system must display products with multimedia information 10 times more than in textual-description-only form (AR2) and the order delivery must be successful 98% of the time (AR4). Finally, there are three optimization *AwReqs*: minimize the system’s response time (AR7), maximize the number of sales (AR5), and minimize the operational cost (AR6). The thresholds of the reference goals related to these *AwReqs* are presented in Table 5, whereas the priorities of their associated indicators are presented in Table 6.

To control the indicators of our system, we elicited five control parameters. The number of virtual servers (*NoS*) deployed that host the content of the E-Shop website and the number of cores

Table 5. Reference Goals

<i>AwReq</i>	$y^\circ(\cdot)$
AR1	$y_1^\circ(\cdot) = 100$
AR2	$y_2^\circ(\cdot) = 10$
AR3	$y_3^\circ(\cdot) = 0$
AR4	$y_4^\circ(\cdot) = 98$
AR5	$y_5^\circ(\cdot) = 0$
AR6	$y_6^\circ(\cdot) = 0$
AR7	$y_7^\circ(\cdot) = 0$

Table 6. Indicator
Priorities

Indicator	Priority
<i>I1</i>	0.1
<i>I2</i>	0.05
<i>I3</i>	0.15
<i>I4</i>	0.1
<i>I5</i>	0.05
<i>I6</i>	0.15
<i>I7</i>	0.45

Table 7. Control Parameter
Weights

Control Parameter	Weight
<i>NoS</i>	1
<i>NoC</i>	0.8
<i>NoAds</i>	1.2
<i>Cr</i>	0.6
<i>VP1</i>	0.8

(*NoC*) assigned to each of them are two of these parameters. The response time, the operational cost, and the probability of failing to load the website's page are highly influenced by *NoS* and *NoC*. The response time is also influenced by the number of ads (*NoAds*) that are displayed on the webpage, since they pose a performance overhead. Another control parameter is the major courier company (*Cr*) the E-Shop chooses for assigning the 75% of its orders, whereas the remaining 25% are equally distributed to other companies. In our application, we have three courier companies, each of them having a different level of reliability as it concerns the success of delivery and different cost (the higher the cost, the more reliable the courier is). Our last control parameter is *VP1*, i.e., the choice between showing multimedia content or only textual information for the product description. Using multimedia content continuously might guarantee the satisfaction of AR2, but when the website is under stress, it has a negative impact on the response time. Table 7 presents the weights of the control parameters derived during the tuning phase of the controller. Finally, we elicited *EvoReqs*, presented in Table 8, for relaxing the threshold of AR4 during holiday periods, when all courier companies are dealing with high workload and strengthened it back to its initial value when these periods ended.

Table 8. *EvoReqs* Operations

<i>AwReq</i>	<i>EvoReq</i> operation
AR4	1. Relax(<i>AR4</i> , <i>AR4'</i> _90) 2. Strengthen(<i>AR1</i> , <i>AR1'</i> _98)

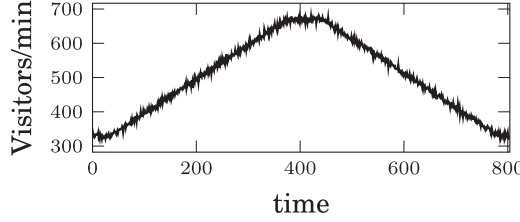


Fig. 9. Active visitors per minute for the performed experiments.

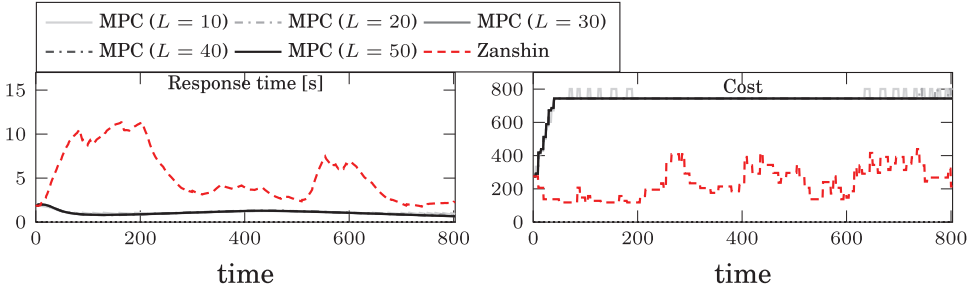


Fig. 10. Indicator measured values for the E-Shop exemplar.

For our evaluation, we run twice our E-Shop for 800 simulation minutes with simulated traffic as presented in Figure 9. The first time we applied *CobRA* and the second time *Zanshin*. We monitored AR6 and AR7, which are related to the optimization goals Minimize Cost and Maximize Performance, respectively. The control parameters used for optimizing these goals are *NoS*, *NoC*, *NoAds*, and *VP1*, which take the value zero if textual mode is selected or the value one if multimedia content is served. In addition, the virtual servers require 2 to 3 minutes for deploying. There are two *AdReqs* that constrain our problem: (1) $NoS \geq 4$ and (2) $NoC \geq 8$. In Figure 10 are presented the output results of both frameworks for each goal and in Figure 11 the corresponding input values. The indicator of AR7 is measured through the system's response time (milliseconds), whereas the one of AR6 is measured through the operational cost (dollars/minute). Table 6 shows that the system's performance is more important than the cost, and therefore, *CobRA*, when the traffic increased, reacted immediately by adding more servers and cores while removing ads and switching to textual mode. On the other hand, *Zanshin* failed to deal with the increased traffic since it tried interchangeably to minimize cost and response time and as a result adaptation actions at each step canceled the effect of adaptations in the previous one. Our observations are also confirmed by Figure 12, where the cost function has considerably lower value in the case of MPC compared to *Zanshin*.

During our trials we did not succeed in identifying an accurate model that would include all the inputs and outputs of our system. The reason is that some goals, and consequently the related output, such as the success rate of order deliveries (AR4) and the maximization of customer satisfaction (AR3), vary much more slowly than response time and cost. In our future work, we plan to

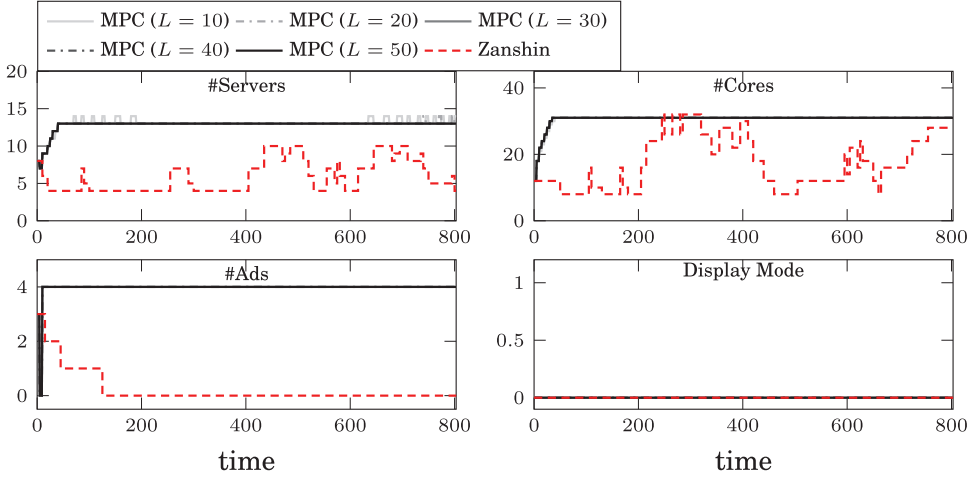


Fig. 11. Control parameter values for the E-Shop exemplar.

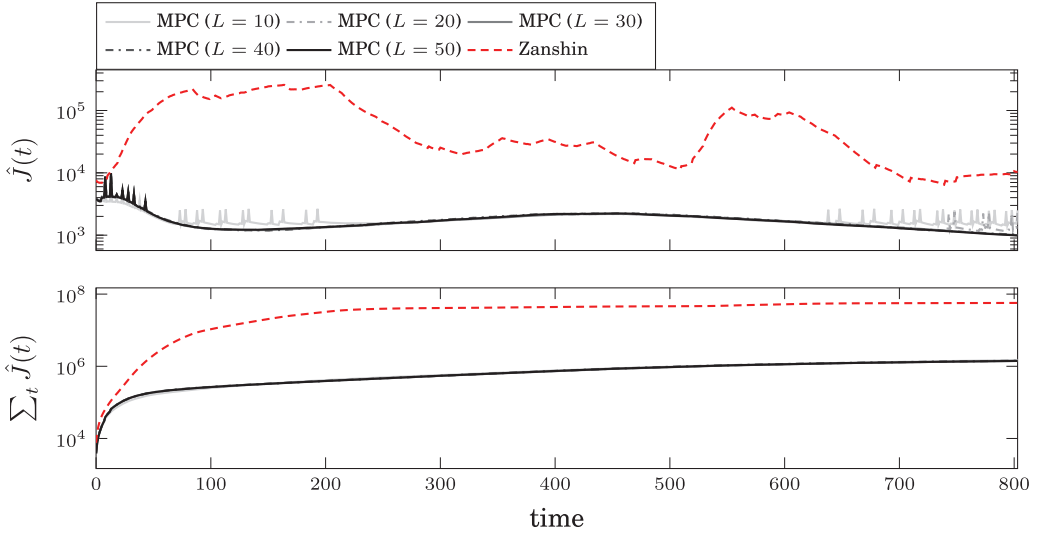


Fig. 12. Adaptation cost for the E-Shop exemplar associated with the cost function J for the considered adaptation strategies. The top graph shows the instantaneous cost, while the bottom graph shows the cumulative cost.

deal with this problem by grouping the goals based on how fast their values change and construct separate controllers for each group. Each controller of course needs to be aware of the adaptation actions produced by the others. This type of control is known as Hierarchical Control (Findeisen et al. 1980).

5.4 Discussion

From the experimental results, we can safely claim that *CobRA* can produce adaptation plans that allow the system to recover faster from failures while maintaining an equilibrium among conflicting requirements. Moreover, our framework outperforms the qualitative adaptation of *Zanshin* in

most cases and confirms that Control Theory can be applied to generic software systems such as the Meeting Scheduler exemplar.

Another notable feature of *CobRA* is that it managed to adapt even if the underlying system had nonlinear behaviours. The simulators we used, in fact, also include nonlinear relationships between inputs and outputs to obtain more realistic behaviors. In practice, most systems have input-output relations that are nonlinear, and therefore, it is important for an adaptation mechanism to handle model imperfections efficiently. In particular, the Kalman filter contributes to correcting the model as the system runs, allowing MPC to make more accurate predictions.

For both our simulations, a linear model was sufficient for predicting the system's behavior. However, this might not always be the case. For systems with nonlinear dynamics, either tailored models can be used (Papadopoulos et al. 2015) or more advanced system identification techniques are available (Ljung 1999, 2010) and nonlinear MPC formulations can be adopted (Allgöwer and Zheng 2000). As future work, we intend to evaluate further our approach using more complex systems, identify their particularities, and apply variations of MPC to deal with them.

The experiments conducted that fed the identification procedure were designed similarly to Filieri et al. (2014); i.e., they required spanning the actuator values over their range. Due to the combinatorial nature, the experiment may take quite long to test all the possible combinations. This issue can be mitigated since it is sufficient to select a subset of the values of the control parameters' domain. In practice, such an approach provides a viable way to perform experiments in a reasonable amount of time. Notice that this issue appears only during the *offline* identification phase, and not when the decision has to be taken at runtime. The same issue appears in other adaptation strategies but at runtime (Moreno et al. 2017, 2015), where the presence of discrete variable makes the formulation of the optimization problem generally NP-hard.

A main drawback of *CobRA* is that it requires an accurate simulation or historical data of the system in order to derive the analytical model it needs to operate. This is not always possible since for software systems there are no methodologies yet, as there are for physical systems, to guide system designers as they simulate a model that can produce data sufficiently similar to those of the real system. It is out of the scope of this article to evaluate the accuracy of a simulated system; metrics that will allow designers to understand how realistic is their simulation are crucial for the applicability of our framework. Therefore, in our future work, we plan to investigate new methodologies for producing realistic simulations for software systems.

Another challenge of our approach is tuning the Kalman filter noise covariances and the MPC cost weights, which is an empirical process. In the literature of Control Theory, this is an open problem as studied by Tran et al. (2014), Ionescu et al. (2016), and Liu and Wang (2000). In addition, the automatic tuning of the controller often depends on the physical laws that describe the model of the system. However, in our work, the model is identified from data produced by a simulation, and is thus just an approximation of the actual dynamics, which is the reason we apply a Kalman filter. The Kalman filter covariance matrix is selected just as an initial guess, but it is adapted online on the basis of the actual behavior of the system. Therefore, the selection of such parameters affects only the initial transient. As it concerns the weights of the optimization problem, we use AHP to elicit initial values and then, while performing trials, we refine them in order to trade off performance and robustness of the controlled system.

Another open challenge for our approach is the automation of the design and implementation of controllers (Tran et al. 2014; Ionescu et al. 2016; Liu and Wang 2000). However, developing methodologies for software engineers, establishing guidelines for tuning the MPC parameters, and facilitating the controller design are part of our future research agenda. Another limitation of *CobRA* is that when not all goals change at the same pace, an accurate analytical model cannot be identified easily and Hierarchical Control is required.

Finally, it is worth noting that some control parameters of our exemplar, such as the number of rooms, are discrete, but according to Equation (1), control parameters are continuous variables. In Control Theory, this problem is known as the actuation design problem. There are two different approaches to handle it: (1) using rounding of the continuous variable computed by the MPC and (2) adopting a Pulse Width Modulation-like policy (Maggio et al. 2012). The second approach, during the period between two adaptations, applies interchangeably the rounded-up and the rounded-down value of the continuous variable, which makes it not suitable for our exemplar, and therefore, we adopted the first one.

6 RELATED WORK

In the field of self-adaptive systems, a variety of approaches use optimization techniques in order to accommodate conflicting requirements during the adaptation process. Rainbow (Cheng et al. 2006) uses Utility Theory: the adaptation strategies are defined at design time based on human expertise and the optimal one is selected at runtime to maximize the overutility of the system. In the same line of work, Cámara et al. (2015) propose the use of Probabilistic Model Checking in order to compose strategies dynamically. Our approach differs from theirs as they require precise knowledge of how each control parameter of the system influences its output, such as that adding one server improves response time by 1 second. In systems with dynamic environments, such relations might change over time and are not always linear. *CobRA*'s MPC uses the derived analytical model to reason about the impact of changing a control parameter instead of human experience and overcomes nonlinearities by applying a Kalman filter.

Zoghi et al. (2014) propose the use of Search-Based Optimization. This approach, similarly to ours, provides a set of control parameters elicited using a goal model. Then, control parameters that affect nonfunctional requirements in the same manner are ranked by the designers of the system and the controller uses a search algorithm to find a solution that maximizes the total obtained utility. Human expertise can contribute to improving software adaptation by making optimal selections. However, the use of analytical models such as the one *CobRA* uses allows better precision and performance of the adaptation process.

Sykes et al. (2010) assign utility properties to all components of the system. Then, based on the component availability, the satisfaction of nonfunctional requirements, and the component dependencies, the adaptation mechanism selects an architectural configuration. Compared to our work, the dependencies among control parameters can be modeled as constraints in the optimization problem that has to be solved by the MPC controller each time the system must adapt.

Another requirements-based and control-theoretic approach is presented in Chen et al. (2014). In this work, the authors propose the use of a PID controller that finds a different configuration over a goal model that captures the system requirements. A SAT-solver is used to find the best configuration based on goal preferences. When soft goals are not met, the controller tunes the values of the assigned preferences so the SAT-solver can find a better configuration. Even though the approach offers an interesting way to control the goal selection process, the output evaluation is limited to a range of values *Satisfied-Denied*. In our approach, we monitor the success of our requirements using *AwReqs* and indicators that provide measurements of higher granularity and allow more precise adaptation.

Recently, an automated solution to introduce the use of control for software systems in a seamless way was proposed in Filieri et al. (2014). This solution treats Single Input and Single Output (SISO) systems by varying a single input and measuring the output. The solution builds on a simple and qualitative dynamic model, which is identified online. More precise yet complicated models can be used at the cost of a higher overhead at runtime (Aleti et al. 2013). The solution in Filieri et al. (2014) works only for SISO systems, while the case of Multiple Inputs and Multiple Outputs

(MIMO) cannot be addressed within that framework. A possible way to deal with MIMO systems is treated in Filieri et al. (2015), where the MIMO control is obtained as an automated synthesis by composing SISO controllers in a hierarchical way. The approach presented in Filieri et al. (2015) is a more modular approach with respect to the one proposed in this article. However, it has the limitation that the influence of different control parameters on the indicators is not included in the model and it is treated only by coupling a single control parameter to a specific indicator. The approach presented in this article includes all the mutual influences in the single model used for the control design. This can be exploited when deciding the values of the control parameters in order to obtain a better adaptation plan.

Additionally, Shevtsov and Weyns (2016) formulate the conversion of the continuous references into discrete settings as a linear optimization problem. The proposed approach avoids partitioning actuators into disjoint sets and allows actuation to minimize a cost function (e.g., considering the priority of different actuators), but it does not provide an explicit means for handling conflicting goals when the satisfaction of one makes others infeasible.

Finally, in the domain of Cloud Computing, variations of MPC have been applied extensively. Gaggero and Caviglione (2016), Kusic et al. (2009), and Lakew et al. (2017) apply look-ahead control to improve the energy consumption and the performance of the cloud. Similarly, in Ghanbari et al. (2014), MPC is applied to improve the replica placement mechanism and deal with multiple service-level objectives. These approaches offer significant improvements to their respective applications, although they are highly customized to the specific problem they are solving. On the other hand, our approach is more generic and therefore easier for software engineers that have no expertise in Control Theory to use it. Moreover, in our work, we integrate control design and requirements engineering in order to provide a guideline on how to integrate MPC with the development of self-adaptive software.

7 CONCLUSIONS

The main contribution of this article is to adopt the concept of MPC for the design of self-adaptive software systems. To accomplish this, we propose a framework, named *CobRA*, that integrates MPC components with previous work on software engineering for self-adaptive systems. We also provide guidelines on how to tune the variables of the MPC controller for better results during the adaptation process.

The distinct feature of *CobRA* compared to other approaches is the use of an analytical model to capture the relationship between the control parameters and the output of the system. This model can accurately predict the system's behavior and allows *CobRA* to react to environmental changes and dynamically compose adaptation plans. The analytical model is the product of an automated system identification process, capturing relations that human experts might not be aware of. We evaluated *CobRA* using two simulations, one for the Meeting Scheduler exemplar and one for an E-Shop, to compare it with the *Zanshin* framework. The results of our evaluation show that control-theoretic concepts can be very effective in producing adaptation plans for software systems and most of the time provide better results than human-experience-based approaches.

Our approach needs to be further evaluated with more and larger case studies and compared with more adaptation frameworks other than *Zanshin*. Moreover, we plan to apply Hierarchical Control in order to overcome difficulties of controlling goals that their level of satisfaction changes at a different rate.

ACKNOWLEDGMENTS

This work has been supported by the ERC advanced grant 267856 Lucretius: “Foundations for Software Evolution” (April 2011–March 2016, <http://www.lucretius.eu>). Alessandro Vittorio

Papadopoulos is funded by the Swedish Foundation for Strategic Research under the project “Future Factories in the Cloud (FiC)” with grant number GMT14-0032. Vítor E. Silva Souza is currently funded by Brazilian research agencies FAPES (# 0969/2015) and CNPq (# 485368/2013-7, # 461777/2014-2).

REFERENCES

- Aldeida Aleti, Barbora Buhnova, Lars Grunske, Anne Kozirolek, and Indika Meedeniya. 2013. Software architecture optimization methods: A systematic literature review. *IEEE Trans. Softw. Eng.* 39, 5 (2013), 658–683. DOI: <http://dx.doi.org/10.1109/TSE.2012.64>
- Frank Allgöwer and Alex Zheng. 2000. *Nonlinear Model Predictive Control*. Vol. 26. Birkhäuser Basel. DOI: <http://dx.doi.org/10.1007/978-3-0348-8407-5>
- Konstantinos Angelopoulos, Alessandro Vittorio Papadopoulos, and John Mylopoulos. 2015. Adaptive predictive control for software systems. In *Proceedings of the 1st International Workshop on Control Theory for Software Engineering (CTSE'15)*. ACM, New York, 17–21. DOI: <http://dx.doi.org/10.1145/2804337.2804340>
- Konstantinos Angelopoulos, Alessandro Vittorio Papadopoulos, Vítor E. Silva Souza, and John Mylopoulos. 2016. Model predictive control for software systems with CobRA. In *Proceedings of the 11th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS'16)*. ACM, New York, 35–46. DOI: <http://dx.doi.org/10.1145/2897053.2897054>
- Konstantinos Angelopoulos, Vítor E. Silva Souza, and John Mylopoulos. 2014. Dealing with multiple failures in zanshin: A control-theoretic approach. In *9th International Symposium on Software Engineering for Adaptive and Self-Managing Systems, Proceedings (SEAMS'14)*. 165–174. DOI: <http://dx.doi.org/10.1145/2593929.2593936>
- Konstantinos Angelopoulos, Vítor E. Silva Souza, and John Mylopoulos. 2015. Capturing variability in adaptation spaces: A three-peaks approach. In *Proceedings of the 34th International Conference on Conceptual Modeling (ER'15)*. 384–398. DOI: http://dx.doi.org/10.1007/978-3-319-25264-3_28
- Karl Johan Åström and Richard M. Murray. 2010. *Feedback Systems: An Introduction for Scientists and Engineers*. Princeton University Press.
- Yuriy Brun, Giovanna Marzo Serugendo, Cristina Gacek, Holger Giese, Holger Kienle, Marin Litoiu, Hausi Müller, Mauro Pezzè, and Mary Shaw. 2009. Engineering self-adaptive systems through feedback loops. In *Software Engineering for Self-Adaptive Systems*. Springer-Verlag, Berlin. 48–70. DOI: http://dx.doi.org/10.1007/978-3-642-02161-9_3
- Eduardo F. Camacho and Carlos Bordons. 2004. *Model Predictive Control*. Springer London.
- Javier Cámara, David Garlan, Bradley R. Schmerl, and Ashutosh Pandey. 2015. Optimal planning for architecture-based self-adaptation via model checking of stochastic games. In *Proceedings of the 30th Annual ACM Symposium on Applied Computing*. 428–435. DOI: <http://dx.doi.org/10.1145/2695664.2695680>
- Bihuan Chen, Xin Peng, Yijun Yu, and Wenyun Zhao. 2014. Uncertainty handling in goal-driven self-optimization - Limiting the negative effect on adaptation. *J. Syst. Softw.* 90 (April 2014), 114–127. DOI: <http://dx.doi.org/10.1016/j.jss.2013.12.033>
- Shang-Wen Cheng, David Garlan, and Bradley R. Schmerl. 2006. Architecture-based self-adaptation in the presence of multiple objectives. In *Proceedings of the 2006 International Workshop on Self-adaptation and Self-Managing Systems (SEAMS'06)*. 2–8. DOI: <http://dx.doi.org/10.1145/1137677.1137679>
- Anne Dardenne, Axel van Lamsweerde, and Stephen Fickas. 1993. Goal-directed requirements acquisition. *Sci. Comput. Program.* 20, 1 (1993), 3–50. DOI: [http://dx.doi.org/10.1016/0167-6423\(93\)90021-G](http://dx.doi.org/10.1016/0167-6423(93)90021-G)
- Antonio Filieri, Carlo Ghezzi, Alberto Leva, and Martina Maggio. 2011. Self-adaptive software meets control theory: A preliminary approach supporting reliability requirements. In *26th IEEE/ACM International Conference on Automated Software Engineering (ASE'11)*. 283–292. DOI: <http://dx.doi.org/10.1109/ASE.2011.6100064>
- Antonio Filieri, Henry Hoffmann, and Martina Maggio. 2014. Automated design of self-adaptive software with control-theoretical formal guarantees. In *36th International Conference on Software Engineering (ICSE'14)*. 299–310. DOI: <http://dx.doi.org/10.1145/2568225.2568272>
- Antonio Filieri, Henry Hoffmann, and Martina Maggio. 2015. Automated multi-objective control for self-adaptive software design. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering (ESEC/FSE'15)*. ACM, New York, 13–24. DOI: <http://dx.doi.org/10.1145/2786805.2786833>
- Antonio Filieri, Martina Maggio, Konstantinos Angelopoulos, Nicolás D'Ippolito, Ilias Gerostathopoulos, Andreas Berndt Hempel, Henry Hoffmann, Pooyan Jamshidi, Evangelia Kalyvianaki, Cristian Klein, Filip Krikava, Sasa Misailovic, Alessandro Vittorio Papadopoulos, Suprio Ray, Amir M. Sharifloo, Stepan Shevtsov, Mateusz Ujma, and Thomas Vogel. 2017. Control strategies for self-adaptive software systems. *ACM Transactions on Autonomous and Adaptive Systems (TAAS)* 11, 4 (Feb. 2017), Article 24, 31 pages. DOI: <https://doi.org/10.1145/3024188>
- Władysław Findeisen, Frederic N. Bailey, M. Brdys, Krzysztof Malinowski, Piotr Tatjewski, and A. Wozniak. 1980. Control and coordination in hierarchical systems. John Wiley & Sons.

- Mauro Gaggero and Luca Caviglione. 2016. Predictive control for energy-aware consolidation in cloud datacenters. *IEEE Transactions on Control Systems Technology* 24, 2 (Mar. 2016), 461–474. DOI : [10.1109/TCST.2015.2457874](https://doi.org/10.1109/TCST.2015.2457874)
- Hamoun Ghanbari, Marin Litoiu, Przemyslaw Pawluk, and Cornel Barna. 2014. Replica placement in cloud through simple stochastic model predictive control. In *2014 IEEE 7th International Conference on Cloud Computing (CLOUD'14)*. 80–87. DOI : [http://dx.doi.org/10.1109/CLOUD.2014.21](https://doi.org/10.1109/CLOUD.2014.21)
- Pontus Giselsson. 2014. Improved fast dual gradient methods for embedded model predictive control. In *IFAC World Congress*, Vol. 19. 2303–2309. DOI : [http://dx.doi.org/10.3182/20140824-6-ZA-1003.00295](https://doi.org/10.3182/20140824-6-ZA-1003.00295)
- Edward Nicholas Hartley, Juan Luis Jerez, Andrea Suardi, Jan M. Maciejowski, Eric C. Kerrigan, and George A. Constantinides. 2014a. Predictive control using an FPGA with application to aircraft control. *IEEE Trans. Control Syst. Technol.* 22, 3 (2014), 1006–1017.
- Edward N. Hartley, Juan L. Jerez, Andrea Suardi, Jan M. Maciejowski, Eric C. Kerrigan, and George A. Constantinides. 2014b. Predictive control using an FPGA with application to aircraft control. *IEEE Trans. Control Syst. Technol.* 22, 3 (May 2014), 1006–1017. DOI : [http://dx.doi.org/10.1109/TCST.2013.2271791](https://doi.org/10.1109/TCST.2013.2271791)
- Clara M. Ionescu, Dana Copot, Anca Maxim, Eva Dulf, Roxana Both, and Robin De Keyser. 2016. Robust autotuning MPC for a class of process control applications. In *2016 IEEE International Conference on Automation, Quality and Testing, Robotics (AQTR'16)*. 1–6. DOI : [http://dx.doi.org/10.1109/AQTR.2016.7501356](https://doi.org/10.1109/AQTR.2016.7501356)
- Juan L. Jerez, Paul J. Goulart, Stefan Richter, George A. Constantinides, Eric C. Kerrigan, and Manfred Morari. 2014. Embedded online optimization for model predictive control at megahertz rates. *IEEE Trans. Autom. Control* 59, 12 (Dec. 2014), 3238–3251. DOI : [http://dx.doi.org/10.1109/TAC.2014.2351991](https://doi.org/10.1109/TAC.2014.2351991)
- Juan L. Jerez, Eric C. Kerrigan, and George A. Constantinides. 2012. A sparse and condensed QP formulation for predictive control of LTI systems. *Automatica* 48, 5 (2012), 999–1002. DOI : [http://dx.doi.org/10.1016/j.automatica.2012.03.010](https://doi.org/10.1016/j.automatica.2012.03.010)
- Joachim Karlsson and Kevin Ryan. 1997. A cost-value approach for prioritizing requirements. *IEEE Softw.* 14, 5 (Sept. 1997), 67–74. DOI : [http://dx.doi.org/10.1109/52.605933](https://doi.org/10.1109/52.605933)
- Jeffrey O. Kephart and David M. Chess. 2003. The vision of autonomic computing. *Computer* 36, 1 (Jan. 2003), 41–50. DOI : [http://dx.doi.org/10.1109/MC.2003.1160055](https://doi.org/10.1109/MC.2003.1160055)
- Cristian Klein, Alessandro Vittorio Papadopoulos, Manfred Dellkrantz, Jonas Dürango, Martina Maggio, Karl-Erik Årzén, Francisco Hernández-Rodríguez, and Erik Elmroth. 2014. Improving cloud service resilience using brownout-aware load-balancing. In *2014 IEEE 33rd International Symposium on Reliable Distributed Systems (SRDS'14)*. 31–40. DOI : [http://dx.doi.org/10.1109/SRDS.2014.14](https://doi.org/10.1109/SRDS.2014.14)
- Mayuresh V. Kothare, Venkataramanan Balakrishnan, and Manfred Morari. 1996. Robust constrained model predictive control using linear matrix inequalities. *Automatica* 32, 10 (1996), 1361–1379. DOI : [http://dx.doi.org/10.1016/0005-1098\(96\)00063-5](https://doi.org/10.1016/0005-1098(96)00063-5)
- Dara Kusic, Jeffrey O. Kephart, James E. Hanson, Nagarajan Kandasamy, and Guofei Jiang. 2009. Power and performance management of virtualized computing environments via lookahead control. *Cluster Comput.* 12, 1 (2009), 1–15.
- Ewnetu Bayuh Lakew, Alessandro Vittorio Papadopoulos, Martina Maggio, Cristian Klein, and Erik Elmroth. 2017. KPI-agnostic control for fine-grained vertical elasticity. In *17th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGrid'17)*. 589–598. DOI : [http://dx.doi.org/10.1109/CCGRID.2017.71](https://doi.org/10.1109/CCGRID.2017.71)
- Emmanuel Letier and Axel van Lamsweerde. 2004. Reasoning about partial goal satisfaction for requirements and design engineering. *SIGSOFT Softw. Eng. Notes* 29, 6 (Oct. 2004), 53–62. DOI : [http://dx.doi.org/10.1145/1041685.1029905](https://doi.org/10.1145/1041685.1029905)
- Feng-Lin Li, Jennifer Horkoff, John Mylopoulos, Renata S. S. Guizzardi, Giancarlo Guizzardi, Alex Borgida, and Lin Liu. 2014. Non-functional requirements as qualities, with a spice of ontology. In *2014 IEEE 22nd International Requirements Engineering Conference (RE'14)*. 293–302. DOI : [http://dx.doi.org/10.1109/RE.2014.6912271](https://doi.org/10.1109/RE.2014.6912271)
- Wei Liu and George Wang. 2000. Auto-tuning procedure for model-based predictive controller. In *2000 IEEE International Conference on Systems, Man, and Cybernetics*, Vol. 5. 3421–3426. DOI : [http://dx.doi.org/10.1109/ICSMC.2000.886537](https://doi.org/10.1109/ICSMC.2000.886537)
- Lennart Ljung. 1999. *System Identification: Theory for the User*. Prentice Hall PTR, Upper Saddle River, NJ.
- Lennart Ljung. 2010. Approaches to identification of nonlinear systems. In *2010 29th Chinese Control Conference (CCC'10)*. 1–5.
- Jan M. Maciejowski. 2002. *Predictive Control: With Constraints*. Prentice Hall.
- Martina Maggio, Henry Hoffmann, Alessandro Vittorio Papadopoulos, Jacopo Panerati, Marco Domenico Santambrogio, Anant Agarwal, and Alberto Leva. 2012. Comparison of decision making strategies for self-optimization in autonomic computing systems. *ACM Trans. Auton. Adaptive Syst.* 7, 4, Article 36 (2012), 32 pages. DOI : [http://dx.doi.org/10.1145/2382570.2382572](https://doi.org/10.1145/2382570.2382572)
- Gabriel Moreno, Alessandro Vittorio Papadopoulos, Konstantinos Angelopoulos, Javier Cámara, and Bradley Schmerl. 2017. Comparing model-based predictive approaches to self-adaptation: CobRA and PLA. In *12th International Symposium on Software Engineering for Adaptive and Self-Managing Systems*. 42–53. DOI : [http://dx.doi.org/10.1109/SEAMS.2017.2](https://doi.org/10.1109/SEAMS.2017.2)
- Gabriel A. Moreno, Javier Cámara, David Garlan, and Bradley Schmerl. 2015. Proactive self-adaptation under uncertainty: A probabilistic model checking approach. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering (ESEC/FSE'15)*. ACM, New York, 1–12. DOI : [http://dx.doi.org/10.1145/2786805.2786853](https://doi.org/10.1145/2786805.2786853)

- Alessandro Vittorio Papadopoulos, Martina Maggio, Federico Terraneo, and Alberto Leva. 2015. A dynamic modelling framework for control-based computing system design. *Math. Comput. Model. Dynam. Syst.* 21, 3 (2015), 251–271. DOI : <http://dx.doi.org/10.1080/13873954.2014.942785>
- S. Joe Qin and Thomas A. Badgwell. 2003. A survey of industrial model predictive control technology. *Control Eng. Pract.* 11, 7 (2003), 733–764.
- Nilabja Roy, Abhishek Dubey, and Aniruddha Gokhale. 2011. Efficient autoscaling in the cloud using predictive models for workload forecasting. In *2011 IEEE International Conference on Cloud Computing (CLOUD'11)*. 500–507. DOI : <http://dx.doi.org/10.1109/CLOUD.2011.42>
- Stepan Shevtsov and Danny Weyns. 2016. Keep It SIMPLEX: Satisfying multiple goals with guarantees in control-based self-adaptive systems. In *Proceedings of the 2016 24th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE'16)*. ACM, New York, 229–241. DOI : <http://dx.doi.org/10.1145/2950290.2950301>
- Vítor E. Silva Souza, Alexei Lapouchnian, Konstantinos Angelopoulos, and John Mylopoulos. 2013. Requirements-driven software evolution. *Comput. Sci. - R&D* 28, 4 (2013), 311–329. DOI : <http://dx.doi.org/10.1007/s00450-012-0232-2>
- Vítor E. Silva Souza, Alexei Lapouchnian, and John Mylopoulos. 2011. System identification for adaptive software systems: A requirements engineering perspective. In *Proceedings of the 30th International Conference on Conceptual Modeling (ER'11)*. 346–361. DOI : http://dx.doi.org/10.1007/978-3-642-24606-7_26
- Vítor E. Silva Souza, Alexei Lapouchnian, and John Mylopoulos. 2012. Requirements-driven qualitative adaptation. In *On the Move to Meaningful Internet Systems (OTM'12)*, Robert Meersman, Hervé Panetto, Tharam Dillon, Stefanie Rinderle-Ma, Peter Dadam, Xiaofang Zhou, Siani Pearson, Alois Ferscha, Sonia Bergamaschi, and Isabel F. Cruz (Eds.). Lecture Notes in Computer Science, Vol. 7565. Springer, Berlin, 342–361. DOI : http://dx.doi.org/10.1007/978-3-642-33606-5_21
- Vítor E. Silva Souza, Alexei Lapouchnian, William N. Robinson, and John Mylopoulos. 2011. Awareness requirements for adaptive systems. In *2011 ICSE Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS'11)*. 60–69. DOI : <http://dx.doi.org/10.1145/1988008.1988018>
- Daniel Sykes, William Heaven, Jeff Magee, and Jeff Kramer. 2010. Exploiting non-functional preferences in architectural adaptation for self-managed systems. In *Proceedings of the 2010 ACM Symposium on Applied Computing (SAC'10)*. ACM, New York, 431–438. DOI : <http://dx.doi.org/10.1145/1774088.1774180>
- Quang N. Tran, Joni Scholten, Leyla Ozkan, and A. C. P. M. Backx. 2014. A model-free approach for auto-tuning of model predictive control. *IFAC Proceedings Volumes* 47, 3 (2014), 2189–2194. DOI : <http://dx.doi.org/10.3182/20140824-6-ZA-1003.01494> 19th IFAC World Congress.
- Yang Wang and S. Boyd. 2010. Fast model predictive control using online optimization. *IEEE Trans. Control Syst. Technol.* 18, 2 (March 2010), 267–278. DOI : <http://dx.doi.org/10.1109/TCST.2009.2017934>
- Melanie N. Zeilinger, Davide M. Raimondo, Alexander Domahidi, Manfred Morari, and Colin N. Jones. 2014. On real-time robust model predictive control. *Automatica* 50, 3 (2014), 683–694. DOI : <http://dx.doi.org/10.1016/j.automatica.2013.11.019>
- Parisa Zoghi, Mark Shtern, and Marin Litoiu. 2014. Designing search based adaptive systems: A quantitative approach. In *Proceedings of the 9th International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS'14)*. 7–16. DOI : <http://dx.doi.org/10.1145/2593929.2593935>

Received October 2016; revised February 2017; accepted May 2017

Copyright of ACM Transactions on Autonomous & Adaptive Systems is the property of Association for Computing Machinery and its content may not be copied or emailed to multiple sites or posted to a listserv without the copyright holder's express written permission. However, users may print, download, or email articles for individual use.